



Group 5: Super RISCy

Out-of-order RISC-V Processor Final Report

Alex Georgiev
alexniko@umich.edu

Ryan Davis
rzd@umich.edu

Chris Yiu
chrisyiu@umich.edu

Cole Talbot
cotalbot@umich.edu

Abstract—In this paper, we present a 2-way superscalar, out-of-order processor based on the MIPS R10K architecture and RISC-V instruction set. Our design is fully synthesizable and verified for correctness by comparing register and memory writes of a diverse benchmark suite to a reference in-order processor. We evaluate the efficacy of each feature of our processor via hardware counters. Our final design achieves an average cycles per instruction (CPI) of 1.373 with a clock period of 10.0ns, equaling an average time per instruction (TPI) of 13.73ns.

I. INTRODUCTION

Prior to implementing our processor, we profiled our benchmark suite on a conventional in-order processor to motivate which features to incorporate. The collected metrics include opcode distribution, branch patterns, and opportunities for forwarding data from producers one cycle ahead of the consumer, along with forwarding opportunities from older stores to same-address loads. In the following, we begin by summarizing the key takeaways that motivated our design decisions before we analyze the measured benefits of these features in our realized design. We also examine how adjusting different parameters of our design (e.g., the sizes of the reorder buffer and reservation station) impact performance. Next, we describe how the critical path of our processor changed over its development cycle. Finally, we conclude by discussing our testing methodology and project management structure.

A. Profiling Summary

From our initial profiling, we found that around 10% of instructions were branches on average, which led us to implement early branch resolution to avoid the delay of waiting for the branch instruction to become the head of the reorder buffer (ROB). We also analyzed branch patterns by simulating

a global history register and found that roughly half of the programs (mainly short programs) were heavily biased towards backward-taken, forward-not-taken branch patterns. For the longer-running programs, we observed more diverse branch history patterns. For this reason, we implemented a tournament branch predictor in our processor that selected between a local predictor and a gshare predictor. We also found that on average, 18% of executed instructions required forwarding data from a producer one cycle ahead of the consumer. Additionally, nearly every program (78%) had a 1-cycle forwarding rate of at least 10%, which motivated our inclusion of early tag broadcast (ETB). Section II challenges the conclusions we drew from our benchmark profiling by: providing estimates for the amount of wasted work saved by incorporating early branch resolution (EBR), the proportion of time each branch predictor was used, and the proportion of time data was forwarded from the common data bus (CDB), among other metrics.

B. Design Summary

We successfully incorporated all of the features we proposed at the inception of the project, with the exception of a return address stack (RAS). Although we spent some time developing a functional RAS, it was late in the project and after observing the minimal performance gain from its initial implementation, we thought our time was better spent elsewhere. A figure of our high-level architectural design is shown in Fig. 1.

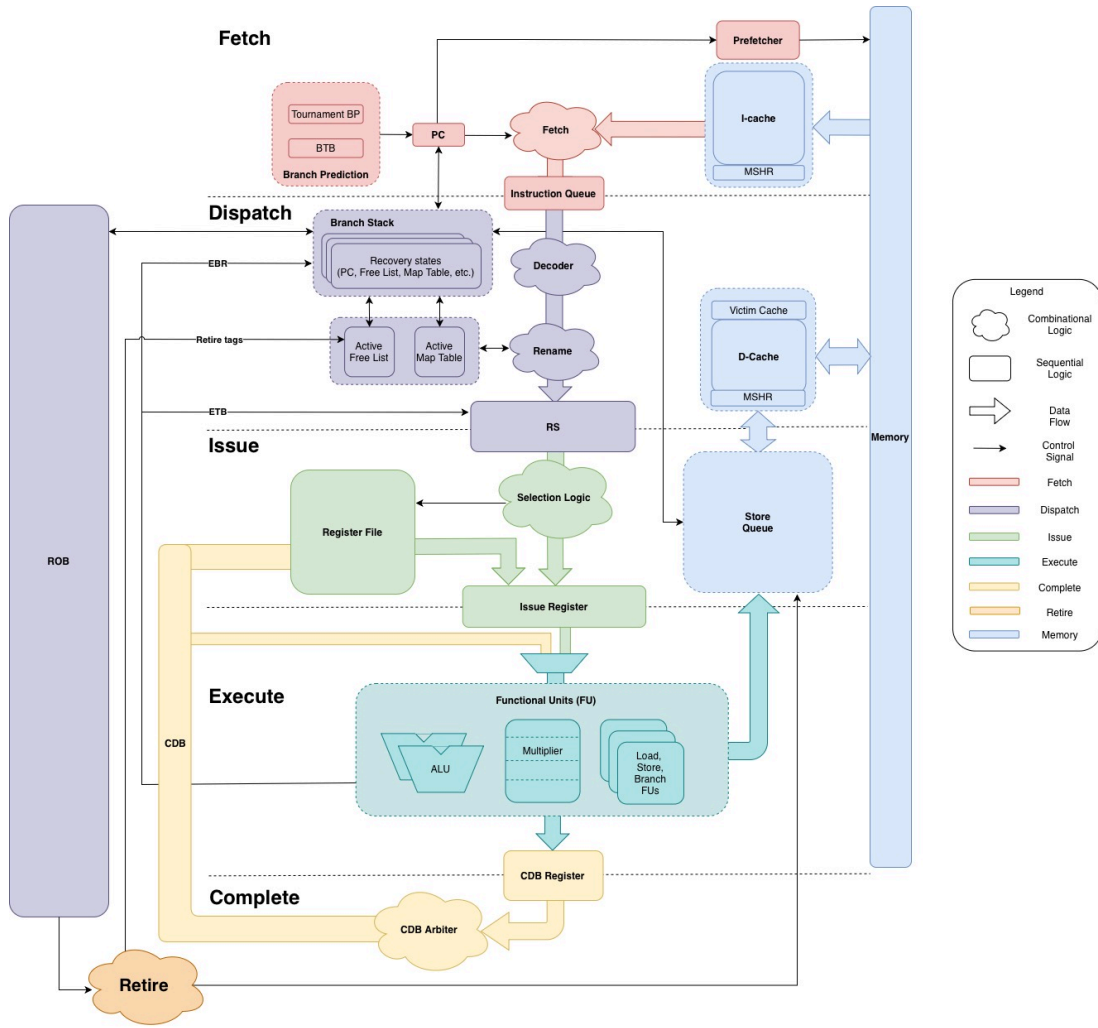


Fig. 1. Architectural diagram of our final processor design. Our processor supports EBR by restoring microarchitectural state (PC, ROB/SQ tail pointers, map table/free list checkpoints) from the branch stack one cycle after a branch has been resolved by an ALU in execute. Instructions that produce a value broadcast their tag on the CDB two cycles early (ETB). This wakes up consumers in the RS that then forward the data from the CDB two cycles later. Our functional units include two ALUs, a 4-stage multiplier, a store unit for address calculation and sending the data to the SQ, and a 2-stage pipelined load unit with the first stage to calculate the address and the second stage for searching the SQ or sending a request to the data cache.

All of the features of our processor are succinctly listed below.

- 2-way superscalar
- Early tag broadcast
- Early branch resolution
- Dual-banked, non-blocking instruction cache
- Instruction prefetching
- Non-blocking, write-back data cache
- Write-back buffer
- Tournament branch predictor
- Byte-level, store-to-load forwarding
- Victim data cache
- Fetch buffer

The parameters of our final design are listed below.

- Clock period: 10ns
- ROB entries: 32
- RS entries: 10
- Store queue entries: 8
- Multiplier stages: 4
- Branch stack size: 4
- BTB size: 64
- Branch predictor entries: 1024

II. ADVANCED FEATURE RESULTS

This section explores the efficacy of the processor's advanced features. It documents the motivation for each feature, including relevant findings from our initial benchmarking. Then we detail the implementation of each feature and the performance benefits when run on our test suite of programs.

A. 2-Way Superscalar Execution

To reduce the CPI of our design, we expanded the pipeline to support 2-way superscalar execution. This allows each stage of the processor to process up to two instructions simultaneously. Notable exceptions are the D-cache and store queue (SQ), which can only process one instruction at a time due to critical path limitations. This yields at least a 1.18x speedup over a 1-way version of the design (methodology for calculating speedup can be found in appendix).

B. Early Tag Broadcast

Early tag broadcast (ETB) allows us to broadcast the destination tag of a producer instruction before its data is ready so that its consumer(s) can start issuing early. This reduces bubbles between dependent instructions.

We implemented this by broadcasting the producer’s destination tag early (before the data is ready) on cycle N, which wakes up the consumer in the RS. On cycle N+1, issue selects the consumer instruction. On cycle N+2, the data is broadcast on the CDB and forwarded into the EX stage where the consumer is. This enables the processor to significantly reduce bubbles between dependent instructions.

We decided to use CDB arbitration that always gives highest priority to load instructions due to long memory latency. Other modules must make a request to the CDB arbiter, and if granted, their tag is broadcast early, which is followed by their data two cycles later. If not granted, they stall and wait for a spot on the CDB to become available. There was CDB contention on 2.7% of cycles, of which 2.3% resulted in an ALU unit being stalled because they have lowest priority in the arbiter.

Early tag broadcast resulted in a 1.20x speedup. 23.2% of all operands in the RS were woken up via ETB, saving cycles between dependent instructions.

C. Early Branch Resolution

The advantage of EBR is that recovering from a mispredicted branch can begin immediately when the branch is resolved, rather than waiting for the mispredicted branch to become the head of the ROB. We evaluate the effectiveness of this feature by estimating 1) the delay of waiting for the branch to reach the head of the ROB and 2) the amount of wasted work saved by recovering earlier.

It is challenging to measure the exact number of cycles saved by EBR, so we use the number of instructions between the ROB head and the mispredicted branch as a proxy for the additional delay if there had not been EBR. We obtain a lower bound estimate for the delay in cycles of waiting for the mispredicted branch to reach the head of the ROB by dividing the number of instructions between the ROB head and mispredicted branch by the superscalar width of our processor (2). Similarly, we count the number of instructions between the mispredicted branch and the ROB tail to estimate the amount of speculative work at the time of the misprediction. We estimate the amount of wasted work by taking the minimum of these values, as shown in Equation 1. This is because speculative work is only wasted for the amount of time it would take the branch to reach the head of the ROB,

provided sufficient speculative work exists to fill that window. Our model is intended to capture the number of cycles that resources are wasted on squashed instructions for a scheme that would require waiting for the branch to reach the head of the ROB.

$$\begin{aligned} \text{RD (cycles)} &= \left\lceil \frac{I_H}{N} \right\rceil \\ \text{SW (cycles)} &= \left\lceil \frac{I_T}{N} \right\rceil \end{aligned} \quad (1)$$

$$\text{WastedWork (cycles)} = \min(\text{RD}, \text{SW})$$

Equation 1. Our model for estimating the average number of cycles saved by EBR. RD denotes the estimated recovery delay of waiting for the branch to reach the head of the ROB. I_H represents the number of instructions between the ROB head and the mispredicted branch. N is the superscalar width of our processor (2). SW represents the estimated speculative work left in the ROB. I_T denotes number of instructions between the mispredicted branch and the ROB tail.

We also estimate the cycles saved by EBR by setting our branch stack size to one. The results of our simulation are shown in Fig. 2. On average, EBR saves around 619 cycles (4.6% of executed cycles) compared to waiting for the branch to reach the head of the ROB per program. Overall, EBR is estimated to have saved 14,229 / 1,296,850 (1.1%) of executed cycles across all performance benchmarks. Although the average is low, considering our branch predictor has high accuracy (90.4%), EBR still yields an improvement over waiting for the branch to become the head of the ROB.

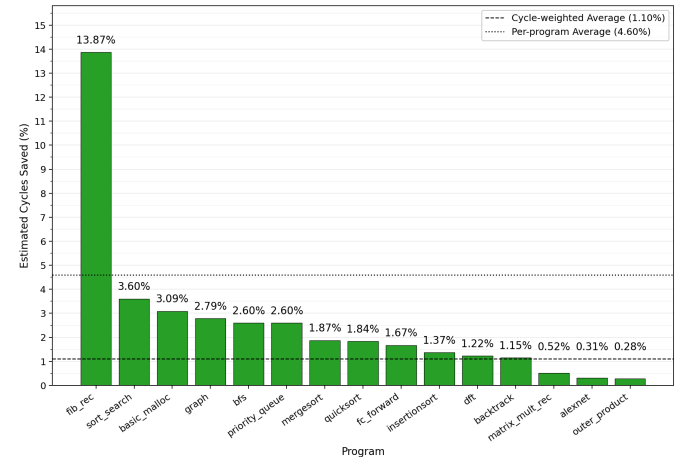


Fig. 2. The estimated amount of cycles saved by EBR as a proportion of the total number of cycles executed. The per-program average and the total cycles saved expressed as a proportion of the total executed cycles are provided for reference.

D. Branch Prediction

For effective branch prediction, we wanted to be able to capture both local and global branch patterns in program execution. To capture both pattern types with one scheme, we

implemented local and gshare predictors with a tournament predictor to pick between them. The local predictor uses a local history table (LHT) and pattern history table (PHT) with 2-bit saturating counters to make predictions. The other predictor is a standard gshare implementation, using an xor of PC and global branch history. The tournament predictor is initialized to weakly favor the local predictor to increase its utilization at the beginning of programs before switching to the gshare once it is warmed up.

The branch predictors work together with the branch target buffer (BTB), which allows us to predict if an instruction is a taken branch, along with its target address before it has been decoded. The BTB is dual-banked so that it can accommodate multiple target address requests in a single cycle. It is updated after branch resolution in EX, where it is notably passed an unconditional branch parameter. The BTB overrides the tournament predictor's output when this flag is high, which ensures all unconditional branches in the BTB are taken. For this reason, unconditional branches do not update any of the branch prediction counters to reduce pollution.

We chose a size of 1024 entries for our branch predictors, which was the optimal size according to our parameter sweeps seen in Fig. 3. Our BTB was limited to 64 entries due to critical path limitations.

Our branch prediction scheme resulted in a 3.25x speedup over a predict not-taken scheme and had an accuracy of 90.4% across 53,321 total conditional branches. The two predictors disagree on 10.4% of predictions. In those cases, the tournament predictor chose gshare on 51.2% of predictions due to its slightly higher accuracy at 89.1% compared to local's 88.6%. At least one predictor was right 94.0% of the time, which shows a potential area for improvement with the tournament predictor.

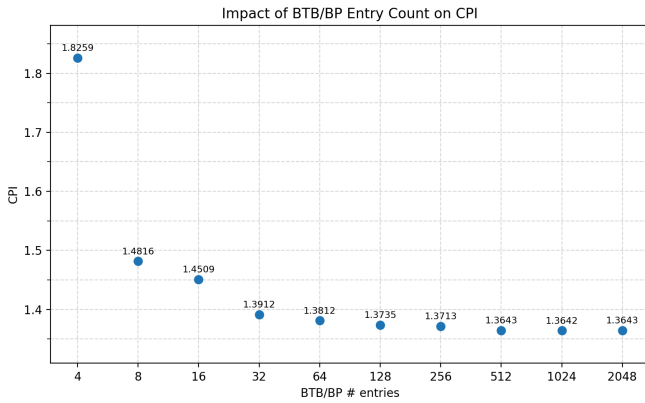


Fig. 3. CPI is significantly higher at smaller entry counts, and reaches an optimal CPI of 1.3642 with an entry count of 1024.

E. Instruction Prefetching

To help reduce backend starvation resulting from long memory latency, instruction prefetching was implemented to optimize fetch engine throughput. The prefetcher uses a four-entry stream buffer, which was the maximum size allowed by the project spec, and prefetches the next four instructions

when the memory arbiter allows. On a mispredict or I-cache miss, the prefetcher clears the buffer and retargets its PC register. On a taken branch, it still redirects its PC but does not clear the buffer. This is because there is a chance it was a misprediction and could need to come back to the blocks fetched on the not-taken path. When an entry hits in the prefetch buffer, it is promoted to the I-cache, which prevents polluting the I-cache with prefetched blocks that were never used. The prefetcher has the lowest priority of any module interacting with memory, so its requests do not affect any other modules.

Instruction prefetching gave us a 1.87x speedup. Of all requests made by the prefetcher, 37.9% ended up being utilized by the fetch engine. The prefetcher combined with the non-blocking I-cache gave a 67.7% hit rate in the fetch engine.

Data prefetching was tested but ultimately not used. Because we were only allowed four prefetch stream buffers, it took away stream buffers from the instruction prefetcher, reducing overall performance.

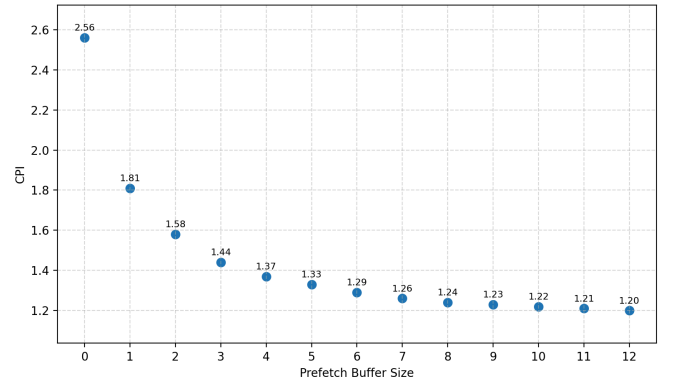


Fig. 4. CPI improves as prefetch stream buffer gets larger. The project specification limited us to four prefetch stream buffer entries.

F. Instruction Cache

We implemented a dual-banked, non-blocking instruction cache (I-cache) with missed status holding register (MSHRs). By making the I-cache dual-banked, the processor can always access two consecutive instructions at a time, even if they do not fall within the same memory block, which importantly matches our superscalar width. We chose our I-cache block size to be the size of one instruction to simplify our combining network.

G. Data Cache

We implemented a non-blocking data cache (D-cache) with MSHRs and configurable associativity. The D-cache has a 4-entry victim cache paired with it that helps reduce conflict misses, which gave a minimal 1.012x speedup. When increasing the number of MSHRs from one to four, we saw a minimal 1.004x speedup. This small improvement could be because our issue logic prioritizes stores over loads in issue and memory, so load instructions do not accumulate very often and cannot take advantage of multiple MSHRs. The structures within our D-cache are parametrized such that the D-cache

can be configured to be direct-mapped (1-way associative), 2-way associative, or 4-way associative. For 2-way and 4-way associativity, we decided to implement a true least recently used (LRU) eviction policy, as power consumption was not a concern. Due to critical path limitations, we chose a 2-way associative design. The I-cache and D-cache hit rates are shown for a subset of the performance programs in Fig. 5, where I-cache hit rate consists of cache hits and prefetch buffer hits and D-cache hit rate consists of cache hits, write-back buffer hits, and victim cache hits.

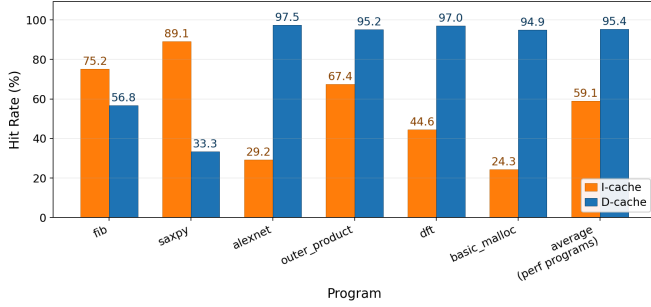


Fig. 5. Across performance benchmark programs, the I-cache had an average 67.7% hit rate, and D-cache had a 95.4% hit rate (1.5% from the victim cache).

III. FOUNDATIONAL STRUCTURE SIZING

In this section, we discuss the impact of structure sizes on performance within foundational modules, and how we chose the final parameters for our processor.

A. Reservation Station

Our reservation station (RS) is shared among all instructions and our issue logic considers all instructions in the RS as candidates for issue. Thus, increasing the size of the RS can create more opportunities for increasing instruction-level parallelism by observing a larger window of instructions. Nonetheless, increasing the RS size may yield diminishing returns, as read-after-write dependencies and structural stalls (e.g., for MSHRs or the CDB) can still prevent two instructions from issuing every cycle. Two instructions issue at the same time on only 17.0% of cycles. Additionally, the fetch stage is limited by memory latency and can only supply so many instructions to the RS at a time. With our final design's size of 10, the RS is only full for 2.5% of cycles. Also, the logic for selecting which instructions to issue is complex, so increasing the RS size can move the critical path to issue selection and require increasing the clock period. As discussed in Section IV, some of the critical paths involved waking up instructions and selecting them for issue.

As shown in Fig. 6, increasing the RS size improves CPI until a size of 16, at which point it saturates. However, designs with RS sizes greater than 10 require increasing the clock period. Although RS sizes of 12 and 16 yield lower CPIs than a design with a 10-entry RS, the increase in clock period required to synthesize these larger structures outweighs the marginal CPI reduction, as shown in Fig. 8. Therefore, the optimal RS size for our design is 10. All designs presented

in our TPI plots use the optimal clock period for synthesizing that structure with respect to TPI rather than simply the smallest synthesizable clock period. For example, it is possible to synthesize an RS of size 16 with a clock period of 10.9. However, it is more beneficial for TPI to synthesize at a clock period of 11.12 for the sake of decreasing the memory latency penalty.

B. Reorder Buffer

The ROB size dictates how many in-flight instructions can be in our processor. To reap the benefit of an increased RS size, our ROB should be sized at least the same size as the RS. However, it may need to be sized larger for further performance gains since already-issued instructions can be stalled on structural hazards like the data cache write port or an MSHR. Similar to the RS, increasing the ROB may increase the critical path of our processor because of the increase in the number of bits required for a physical register tag comparison. After completing the register dataflow aspects of our processor, we found that keeping the RS size constant but increasing the ROB size increased the issue selection critical path for this reason.

As shown in Fig. 7 and Fig. 9, increasing the ROB size improves CPI and TPI, respectively, until a ROB size of 32. Thus, we chose a ROB size of 32 for our final design.

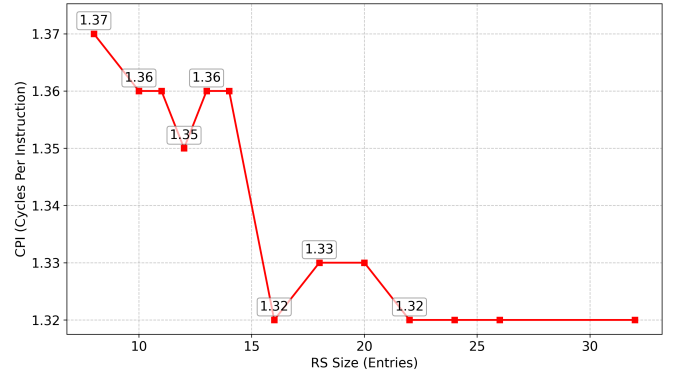


Fig. 6. Increasing the RS size decreases the CPI until a size of 16. Data collected with a 32-entry ROB.

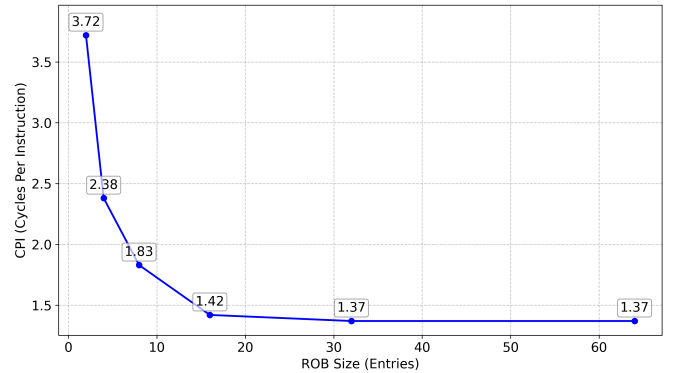


Fig. 7. Increasing the ROB size from 2 to 16 sharply decreases the CPI. However, it saturates at a size of 32. Data collected with RS size = min(ROB, 8).

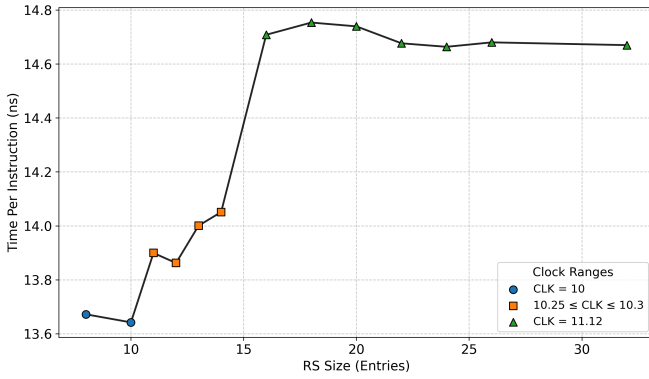


Fig. 8. Increasing the RS size improves performance until a size of 10. Beyond this point, the necessary increase in clock period outweighs any minor reductions in CPI, resulting in worse performance.

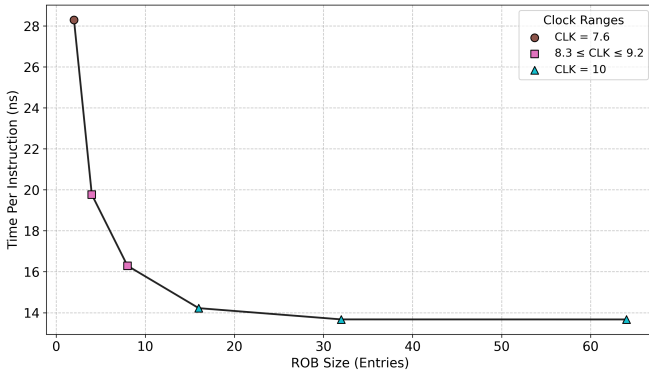


Fig. 9. Increasing the ROB size improves performance until a size of 32, at which point it saturates.

C. Store Queue

We implemented a store queue (SQ) with byte-level, store-to-load forwarding. If all of the bytes are available in the SQ, then the load can be forwarded the data. Otherwise, the load must go to the data cache, or if there are only some bytes, then the load must wait for those stores to be retired (i.e., sent to the memory system). None of the programs in our benchmark suite required stalling a load because only some of the bytes were available in the SQ, so this design choice does not impact our measured performance. Since we only implemented an SQ without a corresponding load queue (LQ), we needed to move the SQ search for ambiguous stores to the issue stage. Every instruction in the RS searches the SQ for older unresolved stores, and if the instruction is a load and it finds an unresolved store, then its request to issue is masked out. This search is necessary in the issue stage since our issue logic does not prioritize instruction age. We found cases when our processor would deadlock due to a combination of read-after-write hazards and memory-age dependencies creating resource cycles.

In our testing suite, only 7.41% of total loads were forwarded their data from the SQ on average, with the remainder fetched directly from the data cache. However, some programs still benefit heavily from the forwarding, as

seen in Fig. 10, where programs like *insertionsort* see rates as high as 29.64%. We measured the performance benefit of store-to-load forwarding by preventing load instructions from issuing if there were any older stores in the SQ. We found that store-to-load forwarding provided a speedup of 12.31%.

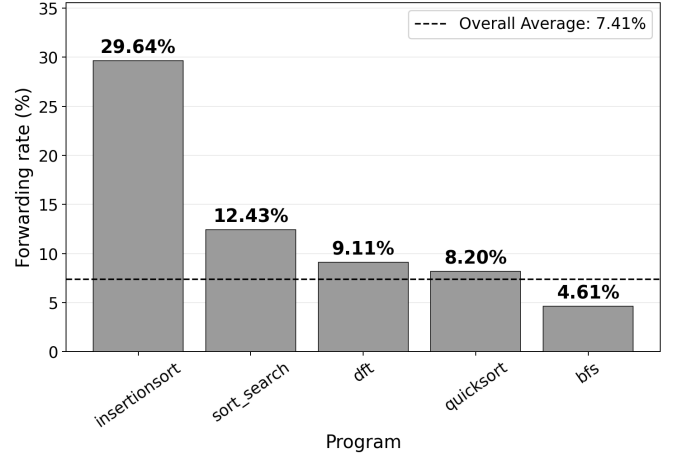


Fig. 10. On average, 7.41% of loads are able to forward their data from older, matching-address stores. There were no cases when a load had to stall because only some of its required bytes were resident in the SQ.

IV. CRITICAL PATH ANALYSIS

After verifying the functionality of our synthesized design, we began exploring opportunities to shorten the critical path of our processor. We analyzed our critical path after completing the register dataflow components of our processor, and again after completing the entire design. The critical path of the processor with only the register dataflow components was the issue selection logic. We reduced the delay of this path from 10ns to 7.5ns by not allowing instructions whose operand tags were currently being broadcast on the CDB to be selected for issue.

The initial critical path of our final design involved calculating the address of a load instruction, checking the SQ to see if it needed to stall in the execute stage, and propagating this signal back to the issue stage to prevent load instructions from issuing. We broke this path by isolating the load address calculation into its own stage, as shown in Fig. 11. The load instruction searches the SQ and data cache the following cycle. This reduced our clock period from 13ns to 11.2ns. However, the address calculation of stores was not similarly pipelined since a store simply needs to write its data into the SQ. Therefore, store instructions still only take one cycle in the execute stage, which avoids an additional CPI penalty.

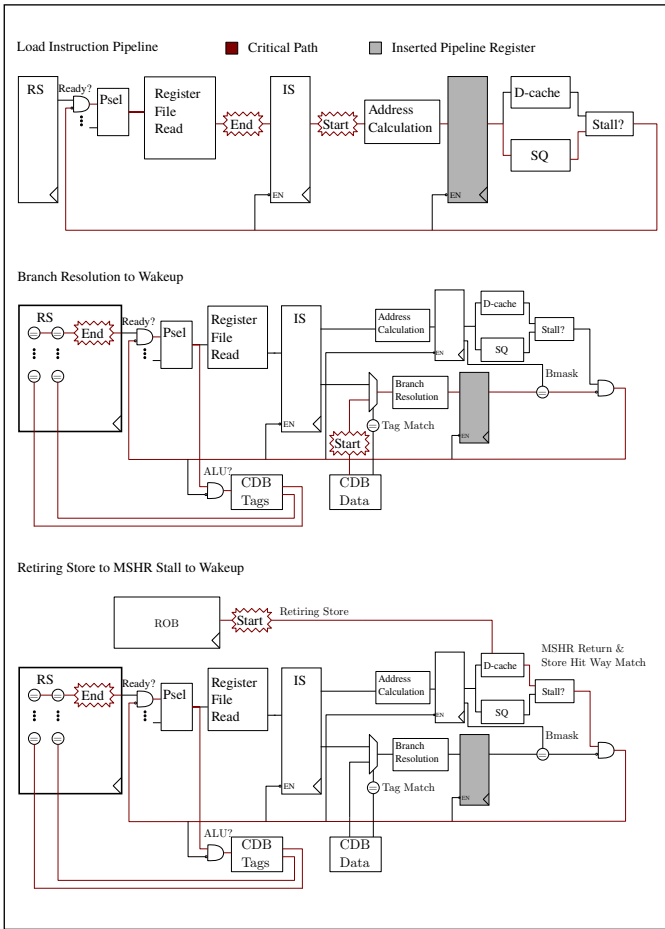


Fig. 11. The first critical path of our processor involved searching the SQ to decide if a load instruction should stall, which masks the request signals of load instructions in issue. The second critical path involved masking the same load signal described above with the mispredict signal, but also continues to wakeup, as ALU instructions need to broadcast their tags on the CDB if they are selected for issue. These paths were broken by inserting pipeline registers. The final critical path also involves this stall signal but originates from checking if a retiring store matched the same data cache way as a returning MSHR entry.

Fig. 11 shows the critical path after pipelining the load functional unit. It begins by resolving a branch instruction that required forwarding data from the CDB. The mispredict signal is then used to mask out a stall signal that indicates if there is a load stalled in the execute stage (i.e., the load is squashed so there is no reason to stall the issue stage). The masked signal propagates back to the issue stage to prevent other memory instructions from requesting to issue. The path ends in wakeup because our processor supports ETB, so if an ALU instruction is selected for issue, then it must broadcast its tags on the CDB. This path was similarly broken by latching the mispredict signal and reduced our clock period from 11.2ns to 10.5ns.

Breaking the mispredict path moved our critical path to checking if a retiring store instruction hit in the same data cache way as a returning MSHR entry, which would require

the data cache to stall any loads in the execute stage. This stall signal from the data cache is used to create the signal for preventing load and store instructions from requesting to issue, as previously discussed. After identifying this path, we applied logic optimizations to shorten it, which brought our final clock period down to 10ns. Applying these optimizations also allowed us to synthesize a 4-stage multiplier and a 10-entry RS, whereas our previous optimal design relied on an 8-stage multiplier and 8-entry RS. The ability to synthesize a larger RS is particularly notable since it lies on our critical path (instruction issue and wakeup). Thus, the benefit of this optimization was larger than the clock-period improvement alone suggests, since it also enabled the use of a larger critical structure that provided an ideal TPI setup for our design.

Although our final clock period is 10ns, the logic optimizations we applied also permitted us to synthesize smaller structures at even lower clock periods (e.g., 9.2ns for a design with an 8-entry ROB and RS). However, we found these designs were not as performant as our final design with the 10ns clock period.

The first optimization in this pass involved improving the issue select logic for load instructions. Our general issue logic does not prioritize instruction age, so we needed to prevent loads younger than the youngest ambiguous store from issuing. This was identified through a deadlock in which a younger load was prioritized over an older load by the priority selector, causing the younger load to stall in execute on an older ambiguous store that was itself a consumer of the older load, preventing the older load from ever issuing. To fix this, we moved our search for an older ambiguous store to the issue stage so that loads would not request to issue if there was an older ambiguous store. Our original SQ search involved complex search logic that checked the entire queue with a priority selector, similar to what was required for checking for stores that could be forwarded data. To reduce this path, we replaced the SQ search logic by latching a pointer to the oldest ambiguous store so that each RS entry just needs to compare its SQ index assigned at dispatch to this pointer.

Building on these timing improvements, the load execution path was optimized by transitioning from sequential to parallel probing of the D-cache and SQ. In our previous design, the D-cache access was gated by the SQ's stall signal, which resided on the critical path. By speculatively querying the D-cache in parallel with the SQ search, the processor effectively hides the latency of the D-cache logic. If the SQ then detects a dependency requiring a stall, the speculative D-cache outputs are simply suppressed.

Finally, a series of synthesis-driven refinements were made to improve timing. We reduced the impact of high fanout global signals by introducing localized staging points, which improved signal distribution and placement. Also, early conditional gating was inserted in long combinational paths so that downstream logic depended only on the relevant cases. Where possible, long if-else and priority chains were replaced with mask-based logic, simplified mux and reduction structures, and collapsed redundant logic patterns identified during timing analysis.

V. TESTING METHODOLOGY

To test our processor and verify it correctly implemented the RISC-V ISA, its program register write-back and final memory outputs were compared to known correct outputs of the processor provided in Project 2 across the suite of provided test programs. These outputs were compared for behavioral RTL simulation as well as for synthesized gate-level simulation. In addition, unit tests were performed on individual modules before integrating into the working processor and micro-benchmarks were run to confirm the functionality of certain advanced features.

A. Test Program Suite Simulation

The provided suite of C and RISC Assembly test programs contains 33 different programs ranging in dynamic instruction count from 1 to 391,760 instructions. By using different compiler optimization flags (O0, O1, O2, O3) for the C programs, a total of 82 test programs were used to evaluate our processor. The golden model processor provided in Project 2 was used to generate correct register write back and final memory outputs for each test program to verify against our processor. Using the *diff* command to compare files, 100% identical register write back and final memory outputs was achieved across all programs when compared to the golden model outputs. Because the processor correctly executes all programs in the diverse testing suite, this demonstrates the processor correctly implements the entirety of the RISC-V ISA while maintaining functional correctness. Correct output was verified through behavioral RTL simulation as well as through synthesized gate-level simulation to ensure our design's physical implementation is also functionally correct and (in the real world) would be ready to be sent to the physical design team.

B. Unit Testing

After implementation, each module was tested individually before attempting to integrate it into the working module. Test benches were created to test for functional correctness, hand-writing test cases for normal operation and corner cases. The test benches had an average overall coverage of 80%, with the SQ test bench obtaining a 93% overall coverage. This allowed for systematic elimination of internal module logic as a root cause during incorrect simulation.

C. Microbenchmarks

Microbenchmarks were used to verify the implementation of advanced features such as ETB, I-cache, and D-cache. To verify ETB, a test program consisting of back to back RAW dependent instructions should result in a CPI closer to 1 than 2, as the dependencies should not create a bubble as a result of pipelining the issue stage. Running this microbenchmark, our processor had a CPI of 0.97, which matches the expected behavior. To verify the I-cache, which has a maximum capacity of 64 instructions, a test program consisting of a loop of 63 instructions should have a near 100% hit rate, considering program overhead to enter the loop. Running this program, the I-cache had a hit rate of 99.91% which matches the expected result. To verify the D-cache, a similar suite of test programs

was constructed of only stores that should result in near 100% hit rate, considering compulsory misses. This program had a D-cache hit rate of 99.90% which matches the expected behavior. These micro-benchmarks allowed us to validate the performance gains associated with our advanced features and provide evidence that the logic functioned correctly. Code for each microbenchmark can be found in section A.2 and A.3 of the appendix.

D. Debug Logs

Comprehensive cycle by cycle debug logs displaying the full contents of each hardware structure in our design were used to debug errors. For each test, any combination of structures could be selected together for debug generation which enabled us to investigate any suspicious interactions between modules. This infrastructure greatly accelerated our ability to quickly identify the origination of an error and observe its resulting incorrect behavior, significantly reducing our overall debug time compared with the experiences we had last semester. An example output displaying a single cycle log of our SQ is displayed in Fig. 12 below.

Idx	H	T	Res	Address	Bytes	Data	Match	Before	C0	C1	C2	C3	Win0	Win1	Win2	Win3
0			1	0x00000000	0001	0x000000a1	1	1	1	0	0	0				
1			1	0x00000001	0010	0x00000200	1	1	0	1	0	0				
2			1	0x00000002	0100	0x00c30000	1	1	0	0	1	0				
3			1	0x00000003	1000	0x00000000	1	1	0	0	0	1				
4	T		0	0x00000000	1111	0x00000000	0	0	0	0	0	0				
5			0	0x00000000	1111	0x00000000	0	0	0	0	0	0				
6			0	0x00000000	1111	0x00000000	0	0	0	0	0	0				
7			0	0x00000000	1111	0x00000000	0	0	0	0	0	0				

Load ADDR: 0x00000000
 Load Posr: 4
 winner idx: 0 | byte[0] = a1
 winner idx: 1 | byte[1] = b2
 winner idx: 2 | byte[2] = c3
 winner idx: 3 | byte[3] = d4
 Store-to-Load Forward
 . [?] winner_idx bytes = (3,2,1,0)
 . [x] reconstructed word = 0xd4c3b2a1
 [DEC04-d0] Passed!

Fig. 12. An example debug log showing the entire SQ for a given cycle. For each entry, the resolved flag, store address, and data value are clearly displayed along with various implementation-level bit masks. The green arrow in the winner columns shows the SQ index whose value is forwarded for a given byte, and the blue arrow shows the oldest ambiguous store position. Below the structure, a byte-level summary is printed for the load.

VI. TAKEAWAYS

A. Future Work

Given the time-constrained nature of this project, there are opportunities for future improvement to continue advancing the performance of our design.

There were multiple features we could have included in a short amount of time with a minimal amount of alterations to our design. One would be the RAS, which as mentioned above was successfully implemented and then scrapped because of its initial low performance impact. With additional time for tuning and testing, we believe this would result in a performance improvement of around 1% based on our profiling of jalr instruction frequency in the testing suite. While not overwhelming, this would have been an easy feature to include by building off of our prior work. A second feature would be to increase the associativity of our I-cache. As shown in

figure Fig. 5, the I-cache accuracy was relatively low despite heavy optimization of our prefetcher and branch prediction schemes. While we experimented with providing the I-cache with some victim cache lines, we observed the structure being too small to make a significant impact.

There are also more substantial opportunities to further improve the design. One of the most important would be adding a load queue (LQ) alongside our existing SQ. A LQ would reduce complexity in the issue stage by moving memory ordering and dependence tracking responsibilities out of issue logic and into a dedicated structure, shortening the amount of memory related logic that must be resolved when instructions are issued. It would also improve our ability to tolerate the 100ns memory latency, since more load misses could be kept in flight and serviced while independent instructions continue executing. Our processor already has the capacity for this optimization due to our non-blocking D-cache, which is currently underutilized (providing only a 0.4% performance increase). This suggests that the limiting factor is not the D-cache’s ability to support multiple outstanding misses, but rather the processor’s limited ability to expose enough memory level parallelism to take advantage of it. As the clock period of our processor decreases, the benefits from a LQ would continue to become more important as the penalty in cycles of long-latency memory accesses increases. It would also enable us to speculatively schedule loads. We believe this would have been advantageous given the low observed store-to-load forwarding rate of 7.41%, meaning that there probably would not be many cases when a rollback would be necessary.

B. Retrospective

Looking back, we have some lessons learned that may have accelerated our design timeline. One important lesson is that building dedicated debug infrastructure earlier would likely have sped up our initial development timeline. Our scheme described in Section V was extremely effective, but we still spent a short amount of time at the beginning of the project using waveforms, and we could have likely saved substantial implementation and testing time, especially before Milestone 1, if we had implemented this strategy sooner.

Second, while benchmarking and upfront research are essential for guiding initial design decisions, it is equally important to remain flexible as more results come out, adapting plans when measured performance does not align with initial expectations. This was experienced firsthand in developing the RAS, where we spent a significant amount of time implementing and debugging an initial design before realizing the performance benefit was minimal. Lastly, we recognized the importance of documenting assumptions about how modules are integrated together. Different people worked on modules that in the end had to interact with each other and we found it was not uncommon that there were misalignments in the assumptions people made about the modules they did not work on. For example, handshaking signals, and how each structure/stage handled mispredict signals. If we had documented these assumptions and interfaces for others to reference in a dedicated place other than code comments,

this would have saved time debugging errors during module integration.

VII. PROJECT MANAGEMENT

A. Gantt Chart

Our original Gantt chart at the beginning of the project can be found at Fig. 17 of appendix A.4. Our final Gantt chart, which shows changes to the plan during the project as well as the actual time it took to complete tasks can be found in Fig. 18 to Fig. 20 of appendix A.4. Between the PDR and Milestone 1, all tasks took the anticipated amount of time with the exception of comprehensive non-memory instruction testing. This included creating a fake fetch engine, integrating all modules for the out-of-order execution of non-memory operations, and debugging issues. In particular, integration and debugging issues resulted in this task taking a week longer than expected. Fortunately, we had allocated spring break as a buffer before Milestone 1, so we were able to meet the deadline without issue.

Between Milestone 1 and the CDR, there were a few changes to tasks, task allocation, and completion timelines. For the memory team, instead of starting with a blocking cache and later transitioning to a non-blocking one with an MSHR, we worked on the D-cache and MSHR in parallel from the start. This allowed us to reuse the working MSHR in the I-cache and integrate with the processor using non-blocking behavior from the beginning, reducing integration overhead. Upon learning more about the memory system in lecture, we also decided to remove the load buffer in favor of a purely SQ design.

For the fetch engine team, the only deviations from the original plan involved the return address stack (RAS) and prefetcher. After implementing a baseline RAS, we were disappointed by its negligible performance benefit. Given this result and the fact that our test programs consisted of only 3% jump-and-link instructions, we decided to scrap the RAS, concluding that our efforts were better spent on the SQ and D-cache. The prefetcher took longer than expected to implement and integrate, mainly due to the complexity of its design and a bug it exposed in the BTB. We also approached the prefetcher with an awareness of its potential for outsized performance gains given our processor’s sensitivity to I-cache misses, so our final implementation includes several additional optimizations beyond our original design.

APPENDIX

A.1 Base Features

We have implemented all required base features. We implemented a base out-of-order (OoO) processor based on the R10K architecture. OoO execution can be verified using a micro-benchmark consisting of a loop of WAR and WAW instructions. This program should have a CPI of approximately 0.5 in a 2-way superscalar OoO processor as register renaming removes the dependencies between instructions, allowing them to be issued in the same and consecutive cycles. Running the OoO micro-benchmark in Fig. 13, our processor had a CPI of 0.58 which matches the expected behavior.

```
/*
 * WAW / WAR microbenchmark for OoO Execution
 * Each 6-instruction tile is only WAW on x4 and WAR on x5 (add reads x5,
 * younger addi overwrites x5). The tail is a 2-instruction WAW on x7.
 */
.section .text
.globl _start

.equ    ITERS, 500

_start:
addi    x1, x0, 0
addi    x2, x0, ITERS
j        loop
/* Fall-through path (not executed); valid NOPs for stray ifetch. */
.rept    61
nop
.endr

/* loop: PC alignment ensures no cache conflicts */
.balign 256
.balign 8
loop:
.rept    10
addi    x4, x0, 1
addi    x4, x0, 77
addi    x4, x0, 42
addi    x5, x0, 100
add     x6, x5, x0
addi    x5, x0, 255
.endr
addi    x7, x0, 1
addi    x7, x0, 42
addi    x2, x2, -1
bnez    x2, loop
wfi
```

Fig. 13. Program consisting of a loop of WAR and WAW instructions, results in a CPI of 0.58. The loop allows instruction fetch latency to be amortized.

We have different functional units with varying latencies outlined below:

- 2 ALUs, 1 cycle
- Store FU, 1 cycle
- Pipelined Load FU, 2 cycles
- Pipelined Multiplier, 4 cycles

We implemented branch prediction with a dual-banked BTB for branch detection and target address prediction, and a tournament predictor for the branch direction.

A.2 Advanced Features

A.2.1 Superscalar execution

We have implemented a 2-way superscalar design. Dispatch, issue, execute, complete (CDB), and retire all have a width of two instructions.

To estimate the speedup of 2-way superscalar design, we modified the processor to be closer to a 1-way design. We changed issue to only select one instruction, and CDB to only grant one request. This is only an estimation, as changing some parts of the pipeline to be 1-way, such as the front-end, would have much higher complexity. Regardless, we measured a speedup of at least 1.18x for going from a 1-way design to a 2-way superscalar design, as referenced in Section II.A

A.2.2 Early Branch Recovery

We have implemented early branch recovery. To approximate speedup from EBR, we changed the branch stack to size 1. This is not a full approximation, as we still are not waiting until the branch reaches the head of the ROB to squash instructions. Regardless, we measured a speedup of 1.041x when going from a branch stack size of 1 to 4.

A.2.3 Early Tag Broadcast

We have implemented early tag broadcast. After optimizing for minimal clock period, tags are forwarded two cycles in advance. A micro-benchmark consisting of a loop of back-to-back RAW instructions, see Fig. 14, results in a CPI of 0.97, verifying early tag broadcast is occurring. If the CPI was closer to 2, this would mean splitting up the issue stage in an attempt to implement early tag broadcast resulted in a one cycle bubble.

```
/*
 * RAW dependency chain packed into an I-cache-friendly loop :
 * 256 B-aligned loop body = 62 back-to-back addi x1,x1,1
 * plus decrement + branch (64 instructions / 256 B).
 */
.section .text
.globl _start

.equ    ITERS, 500

_start:
addi    x1, x0, 0
addi    x2, x0, ITERS
j        loop
/* Fall-through path (not executed); valid NOPs for stray ifetch. */
.rept    61
nop
.endr

/* loop: must land at 0x100 = 256 (see dump / address map). */
.balign 256
.balign 8
loop:
.rept    62
addi    x1, x1, 1
.endr
addi    x2, x2, -1
bnez    x2, loop
wfi
```

Fig. 14. Program consisting of a loop of back-to-back RAW instructions, results in a CPI of 0.97. The loop allows instruction fetch latency to be amortized.

To measure speedup for Early Tag Broadcast, we wired in the CDB data to the RS instead of the early-broadcasted tags. This removes ETB and only wakes up instructions in the RS once the data is ready. We measured a 1.204x speedup for

ETB, as referenced in Section II.B.

A.2.4 Fetch Enhancements

We implemented a tournament branch predictor that chooses between a G-Share predictor and a local 2-bit saturating counter predictor. Branch predictor speedup was determined by setting the predict taken signal to 0 in the fetch engine, giving a 3.249x speedup as detailed in Section II.D.

Our processor also features a FIFO queue, called fetch queue, between fetch and decode stage.

A return address stack was attempted and an initial design was implemented. However, the performance gain was negligible and we ultimately decided to remove the RAS to spend our time more efficiently on other features.

A.2.5 Memory Hierarchy Improvements

We implemented a next-line instruction prefetcher that places instructions into a prefetch buffer of size 4 instructions. To determine the speedup from the prefetcher, the prefetcher's request signal to memory was tied to 0, which prevents it from getting anything from memory. We measured a 1.873x speedup from the prefetcher, as referenced in Section II.E.

We implemented a parametrized associative D-cache. It is parametrized to be 1-way, 2-way, or 4-way associative. Executing the performance benchmark programs, these schemes had a CPI of 1.9018, 1.8915, and 1.8901 respectively.

We implemented a 4-entry victim cache that is attached to the D-cache. To measure speedup from the victim cache, we disconnected it from the D-cache. We measured a 1.012x speedup from the victim cache, as referenced in Section II.G.

We implemented a write-back data cache with write-back buffers, a Store Queue with conservative load scheduling, and byte-level store-to-load forwarding.

A.3 Additional Micro-benchmarks

A.3.1 I-Cache Micro-benchmark

```

/*
 * I-cache microbenchmark: 64-instruction loop (256 B) at a 256 B-aligned PC.
 * Prologue: load iteration count, then jump over a NOP sled so the loop
 * starts at 0x100 (32 distinct 8 B sets, no index wrap within the body).
 */
.section .text
.globl _start

.equ    ITERS, 8888

_start:
    li    x1, ITERS
    j     loop
    /* Fall-through path (not executed); keep valid NOPs for stray ifetch. */
    .rept 61
    nop
    .endr

    /* loop: must land at 0x100 = 256 (see dump / address map). */
    .balign 256
    .balign 8
loop:
    .rept 62
    nop
    .endr
    addi  x1, x1, -1
    bnez  x1, loop
    wfi

```

Fig. 15. Program consisting of a loop of no-op instructions, results in a I-cache hit rate of 99.91%. The loop allows program overhead and instruction fetch latency to be amortized.

A.3.2 D-Cache Micro-benchmark

```

/*
 * D-cache store hit loop (DCACHE_WAYS=1, DCACHE_SETS=32).
 *
 * Sixty-four distinct byte addresses: for each set s in 0..31, stores to
 * BASE + (s<<3) + 0 and +1 – two locations per line, 32 lines total, full
 * occupancy of the cache with no evictions on repeat.
 *
 * First outer iteration: first sb on each line misses; second sb on that line
 * hits. Every sb in the second and later outer iterations hits.
 */
.section .text
.globl _start

.equ    SET_COUNT, 32
.equ    LINE_BYTES, 8

.equ    DATA_BASE, 0x00001000
.equ    ITERS, 500

_start:
    li    x1, DATA_BASE
    li    x2, ITERS
    li    x6, SET_COUNT

outer:
    li    x3, 0          /* set index */

inner:
    slli  x7, x3, 3      /* x7 = set_index * LINE_BYTES */
    add   x4, x1, x7     /* line base address */
    sb    x0, 0(x4)
    sb    x0, 1(x4)
    addi  x3, x3, 1
    bne   x3, x6, inner

    addi  x2, x2, -1
    bnez  x2, outer
    wfi

```

Fig. 16. Program consisting of a loop of store instructions, results in a D-cache hit rate of 99.90%. The loop allows program overhead and instruction fetch latency to be amortized.

A.4 Gantt Charts

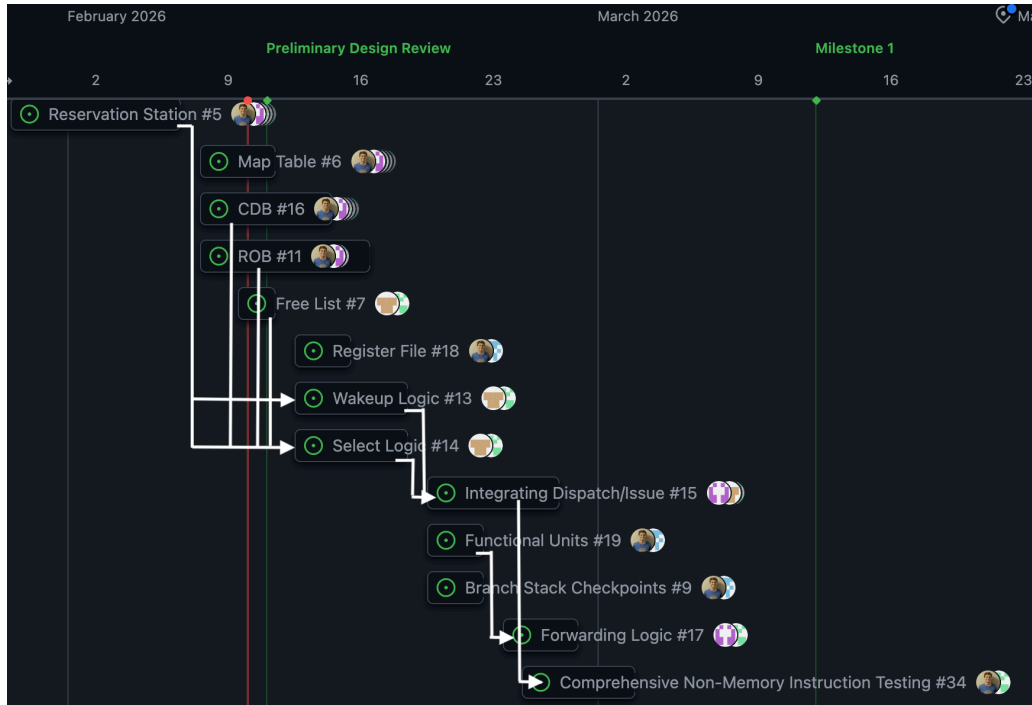


Fig. 17. Initial PDR to Milestone 1 Gantt Chart.

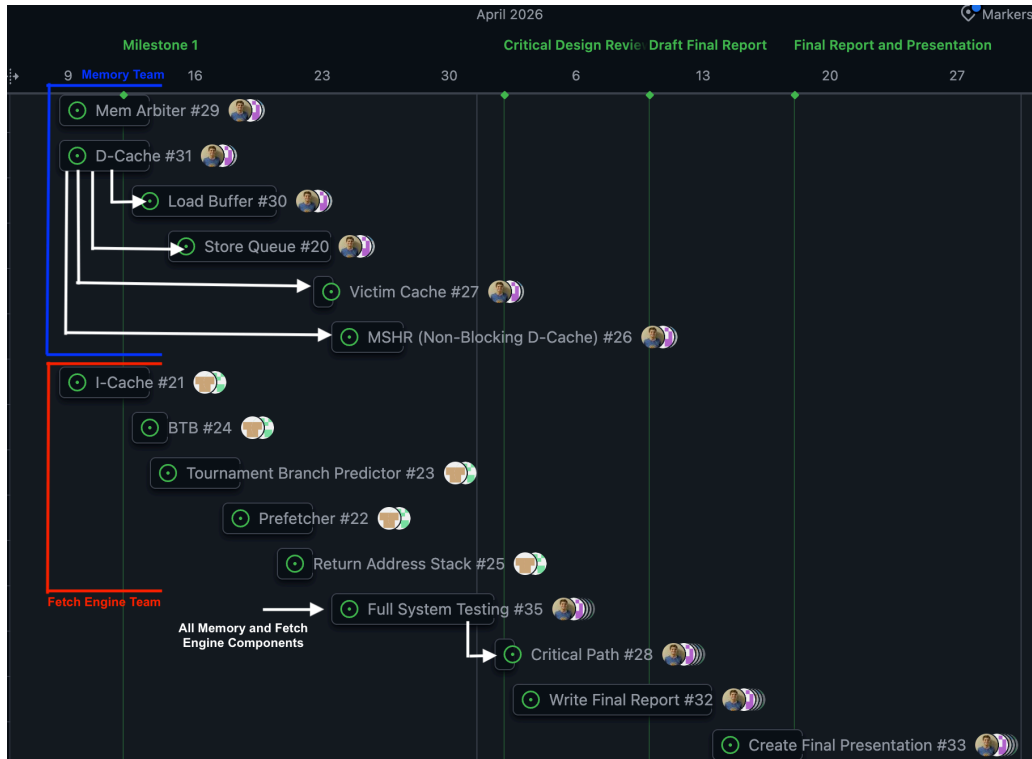


Fig. 18. Initial Milestone 1 to Final Report and Presentation Gantt Chart.

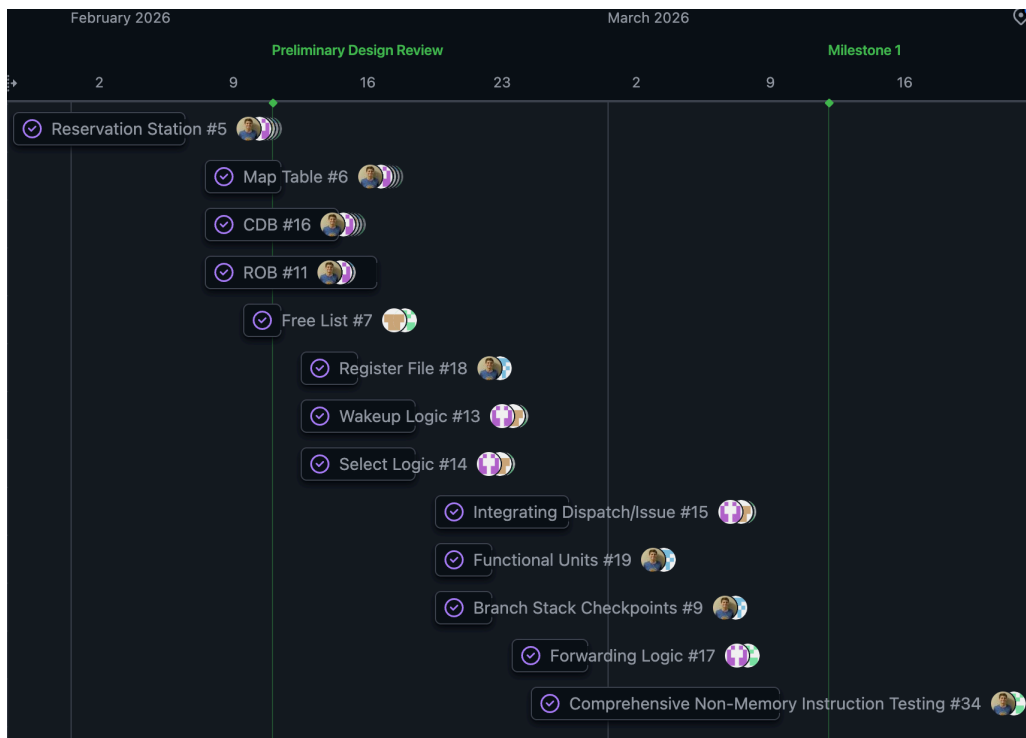


Fig. 19. Actual PDR to Milestone 1 Gantt Chart.

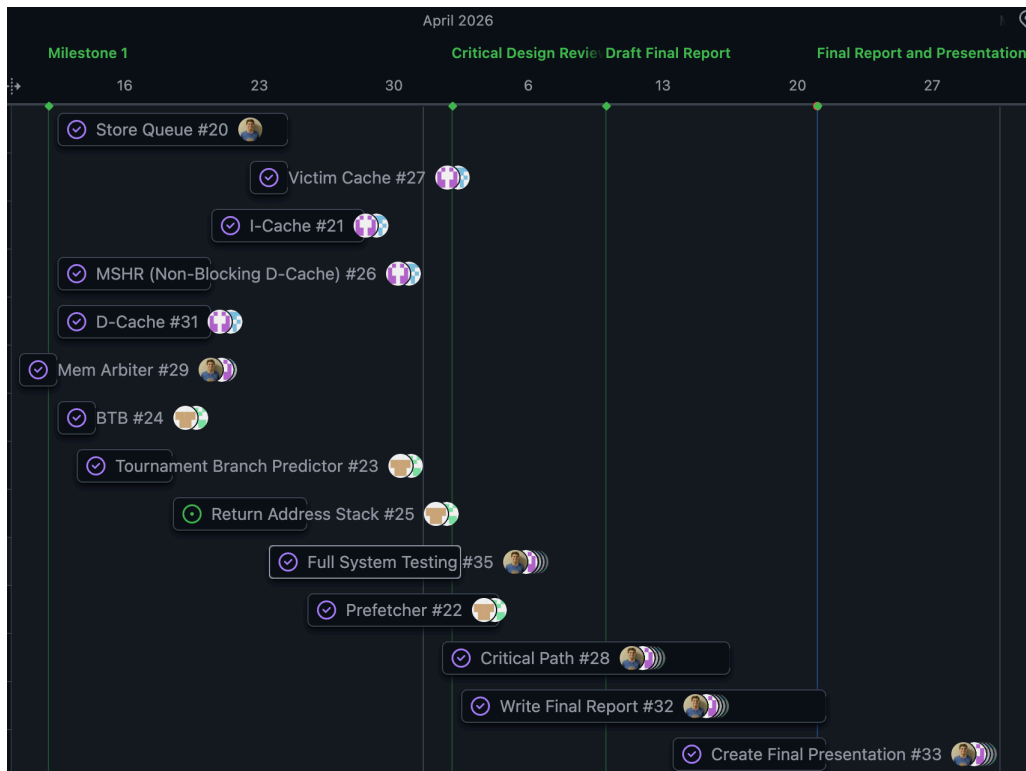


Fig. 20. Actual Milestone 1 to Final Report and Presentation Gantt Chart.